1.Create a simple HTML login form with username and password fields, then write a JavaScript function that simulates a brute force password attack by trying a list of 5 common passwords to guess the correct one.

```
<!DOCTYPE html>
<html>
<body>
 <h2>Login Form</h2>

 <input type="text" id="username"
      placeholder="Username">
 <input type="password" id="password"
      placeholder="Password">

 <button onclick="bruteForce()">Test Passwords</button>
 <p id="result"></p>

 <script>
  function bruteForce() {
   // Demo password stored locally
   let correctPassword = "admin123";

   let passwords = [
    "123456",
    "password",
    "admin",
    "qwerty",
```

```
    "admin123"
  ];

  for (let pass of passwords) {
    if (pass === correctPassword) {
      document.getElementById("result")
        .innerText = "Password Found: " + pass;
      return;
    }
  }

  document.getElementById("result")
    .innerText = "Password Not Found";
 }
</script>
</body>
</html>
```



2.Write a Python script that takes a list of 10 leaked passwords (e.g., 'password', '123456', 'qwerty', etc.) and checks if a user's input password matches any password in the list, simulating a dictionary attack.

```
passwords = [
```

```
    "123456",
    "password",
    "qwerty",
    "admin",
    "12345678",
    "welcome",
    "abc123",
    "iloveyou",
    "admin123",
    "letmein"
]

user_password = input("Enter password: ")

if user_password in passwords:
    print("Password found in dictionary!")
else:
    print("Password not found!")
```



3.Simulate a credential stuffing attack by writing code that takes a list of username-password pairs and checks which pairs successfully log in to a mock app where the correct credentials are stored in a dictionary.

```
# Mock users
```

```
users = {
   "admin": "admin123",
   "user1": "hello123",
   "user2": "pass123"
}

# Test credentials
credentials = [
   ("admin", "123456"),
   ("user1", "hello123"),
   ("user2", "wrongpass"),
   ("admin", "admin123")
]

for username, password in credentials:
   if users.get(username) == password:
      print(username, "- Login Successful")
   else:
      print(username, "- Login Failed")
```



**4.Given a login form, implement a simple account lockout mechanism in JavaScript that locks the user out after 3 failed login attempts, similar to what apps like Paytm or PhonePe do for security.

Hint: Use a counter variable to track failed attempts and disable the login button after 3 tries.**

```
<!DOCTYPE html>
<html>
<body>
 <h2>Secure Login</h2>

 <input type="text" id="username"
      placeholder="Username">

 <input type="password" id="password"
      placeholder="Password">

 <button id="loginBtn" onclick="login()">
  Login
 </button>

 <p id="message"></p>

 <script>
  let attempts = 0;

  function login() {
```

```
 let username =
  document.getElementById("username").value;

 let password =
  document.getElementById("password").value;

 if (username === "admin" &&
    password === "admin123") {
  document.getElementById("message")
   .innerText = "Login Successful!";
  attempts = 0;
 } else {
  attempts++;

  document.getElementById("message")
   .innerText = "Wrong Password!";

  if (attempts >= 3) {
   document.getElementById("loginBtn")
     .disabled = true;

   document.getElementById("message")
     .innerText = "Account Locked!";
  }
 }
}
```

</script>

</body>

</html>

